Register No.															

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
(Common to all branches)

SET - C

Academic Year: 2023-24 (ODD)

Test: CLA-T2 **Date: 2-11-2023**
Course Code & Title: 21CSC203P Advanced Programming Practice **Duration:** 100 minutes
Year & Sem: II Year / III Sem **Max. Marks:** 50

Course Articulation Matrix: ***(to be placed)***

S.No.	Course Outcome	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
1	CO1	3	2	-	-	-	-	-	-	-	-	-	-	-	2	-
2	CO2	3	2	-	1	-	-	-	-	-	-	-	-	-	2	-
3	CO3	3	-	2	-	2	-	-	-	-	1	-	-	-	2	-
4	CO4	3	2	-	1	-	-	-	-	-	-	-	-	-	2	-
5	CO5	3	-	2	1	2	-	-	-	-	1	-	-	-	2	-

	Part – A - (5 x 4 = 20 Marks) Answer any 5 Questions					
1	What is the difference between process-based multitasking and thread-based multitasking? In which scenarios would y Process-based multitasking and thread-based multitasking are two approaches to achieving multitasking in computer systems. They differ in how they manage and execute multiple tasks concurrently. Here's an overview of the differences between the two and when to choose one over the other: 1. Process-Based Multitasking: Processes: In process-based multitasking, each task or job is a separate process, which means it has its own memory space, program counter, and system resources. These processes run independently of each other. Isolation: Processes are isolated from each other, so if one process crashes or encounters an issue, it generally does not affect other processes. Scalability: Creating new processes can be resource-intensive because each process requires a separate memory allocation and operating system overhead. Scenarios for Process-Based Multitasking:	4	L3	1	1	1.6.1

	Isolation: When you need strong isolation between tasks, so that if one task fails, it doesn't affect others. This is important in critical systems like operating systems, web servers, or mission-critical applications. Parallelism on Multi-core CPUs: When you want to take advantage of multi-core processors by running different tasks in separate processes, utilizing the full capabilities of the hardware. 2. Thread-Based Multitasking: Threads: In thread-based multitasking, multiple threads within the same process share the same memory space and system resources. Threads are smaller units of execution within a process. Lightweight: Threads are generally more lightweight to create and manage compared to processes. They can be created and destroyed more quickly. Communication: Threads within the same process can communicate more easily since they share memory. Scenarios for Thread-Based Multitasking: Efficiency: When you need a more efficient way to achieve multitasking, especially on single-core systems or in cases where the overhead of creating separate processes is prohibitive. Shared Data: When tasks need to share data or collaborate closely, using threads within the same process can simplify communication and data sharing. User Interface Applications: Many graphical user interface (GUI) applications use threads for responsive user interfaces, where one thread manages the UI, while other threads handle background tasks. The choice between process-based and thread-based multitasking depends on your specific requirements and the trade-offs you are willing to make. In some cases, a combination of both may be used to take advantage of their respective strengths. For example, a program may use multiple threads for multitasking within a single process and also launch multiple processes for specific tasks that require strong isolation.					
2	What programming languages and libraries are commonly used for concurrent programming, and what features do they offer to facilitate concurrent development?	4	L3	2	2	2.6.4

	Concurrent programming is a crucial aspect of modern software development, as it allows applications to efficiently utilize multi-core processors and handle multiple tasks concurrently. Various programming languages and libraries provide features and tools to facilitate concurrent development. Here are some commonly used programming languages and libraries for concurrent programming and the features they offer: 1. Java: Java Threads: Java provides a built-in thread model using the java.lang.Thread class. Developers can create and manage threads to achieve concurrency. Java Concurrency Utilities: The java.util.concurrent package offers a range of high-level concurrency utilities, such as thread pools, concurrent data structures, and synchronization primitives like Locks and Semaphores. 2. C++: C++ Threads: The C++11 standard introduced the <thread> library, which provides a standard way to work with threads. Developers can use the std::thread class to create and manage threads. C++ Standard Library Concurrency Features: C++ also offers other concurrency features in the <mutex>, <condition_variable>, and <future> headers for synchronization and managing asynchronous operations. 3. Python: Threading and Multiprocessing: Python has a built-in threading module for thread-based concurrency and a multiprocessing module for process-based concurrency. These modules provide easy ways to create and manage threads and processes. Asyncio: Python's asyncio library enables asynchronous, event-driven programming, allowing developers to write non-blocking, concurrent code using coroutines and event loops. 4. Go (Golang): Goroutines: Go is known for its lightweight concurrency model with goroutines. Goroutines are lightweight threads managed by the Go runtime, making it easy to create and manage thousands of concurrent tasks. Channels: Go's built-in channels facilitate communication and synchronization between goroutines, making it easy to					

	coordinate concurrent tasks. 5. Rust: Ownership System: Rust's ownership system enforces memory safety and concurrency safety without the need for a garbage collector. It provides features like ownership, borrowing, and lifetimes to prevent data races and ensure thread safety. Standard Library: The Rust standard library offers concurrency primitives like std::thread, std::sync, and std::mpsc (multi-producer, single-consumer channels) for managing concurrency safely. 6. Erlang: Actor Model: Erlang is built around the actor model, where concurrent tasks are represented as independent actors. Each actor communicates via message passing, providing isolation and fault tolerance. OTP (Open Telecom Platform): Erlang's OTP library provides a framework for building robust, fault-tolerant, and scalable concurrent systems. 7. .NET (C#): Task Parallel Library (TPL): .NET offers TPL for concurrent programming. Developers can use the Task class to represent asynchronous and parallel operations. Asynchronous Programming: .NET supports asynchronous programming with the async and await keywords, allowing developers to write non-blocking code. These programming languages and libraries offer various features to support concurrent programming, including thread management, synchronization mechanisms, and abstractions for parallelism. The choice of language and library depends on your specific project requirements and the level of control and safety you need when working with concurrency.					
3	What are the limitations or challenges of shared memory system? Data Race Conditions: Shared memory systems are susceptible to data race conditions when multiple threads or processes access and modify shared data concurrently without proper synchronization. Data races can lead to unpredictable behavior, crashes, or corruption of data. Synchronization Overhead: To avoid data races, shared memory systems often require synchronization mechanisms	4	L4	1	1	1.6.1

	like locks, semaphores, or mutexes. Implementing and managing these synchronization mechanisms can add complexity to the code and introduce overhead, potentially slowing down the system. Deadlocks: Incorrectly managed locks and synchronization can lead to deadlocks, where two or more processes or threads are stuck, unable to make progress because they are waiting for resources held by each other. Resource Contention: As multiple processes or threads compete for access to shared resources, resource contention can occur, leading to reduced performance and throughput. Complexity: Developing and debugging shared memory systems can be more challenging than other forms of concurrency due to the need for careful management of shared data and synchronization primitives.					
4	What is the role of the bind() function in socket configuration? How do you use it to specify a local address and port for a TCP socket? The bind() function is a critical step in socket programming, particularly when working with TCP sockets. Its primary role is to associate a specific local network address and port with a socket, which means it specifies the endpoint on the local system to which the socket should be bound. This is necessary for both server and client sockets. The bind() function is typically used with Internet Protocol (IP) sockets, such as TCP or UDP. Here's how you use the bind() function to specify a local address and port for a TCP socket: Create a Socket: Before you can use bind(), you need to create a socket using the socket() function. In the case of a TCP socket, you would typically use SOCK_STREAM as the socket type. Define a struct sockaddr_in: You should create a struct sockaddr_in that holds the local address and port information. This structure is used to specify the local endpoint to which the socket will be bound.	4	L4	2	1	1.7.1
5	What is the significance of the listen() function in TCP socket configuration, and what is the maximum number of pending connections it can handle? The listen() function is an important step in the configuration of a TCP socket, specifically on the server side of a network application. Its primary purpose is to make a TCP socket passive, enabling it to accept incoming client connections. The listen() function is typically used in conjunction with the bind() function, which specifies the local address and port to which the server socket should be	4	L3	1	2	2.6.4

	bound. Server Setup: The listen() function is called after a socket is created using the socket() function and bound to a specific local address and port using the bind() function. The socket is configured to listen for incoming client connections. Queue for Incoming Connections: When a server socket is in the listening state, it establishes a queue for incoming connection requests. These connection requests are typically placed in a queue known as the "backlog." Backlog Size: The listen() function allows you to specify the size of the backlog, which is essentially the maximum number of pending connections that can be queued while the server is processing existing connections. The actual maximum number of pending connections a server can handle depends on the operating system and its configuration.					
6	How do symbolic programming languages handle variables and bindings differently from languages in other paradigms, such as procedural or object-oriented languages? Symbolic programming languages, like Lisp, and other paradigms, such as procedural or object-oriented languages, handle variables and bindings differently due to the unique characteristics and goals of symbolic programming. Here are some key differences in how variables and bindings are managed in symbolic programming languages compared to other paradigms: Procedural/Object-Oriented Languages: Many procedural and object-oriented languages use static typing, where variables have fixed types at compile time. This provides type safety but may be less flexible in symbolic manipulation.	4	L5	1	3	2.4.1
7	Can events in a database system be monitored or logged? The listen() function is an important step in the configuration of a TCP socket, specifically on the server side of a network application. Its primary purpose is to make a TCP socket passive, enabling it to accept incoming client connections. The listen() function is typically used in conjunction with the bind() function, which specifies the local address and port to which the server socket should be bound. Server Setup: The listen() function is called after a socket is created using the socket() function and bound to a specific local address and port using the bind() function. The socket is configured to listen for incoming client connections. Queue for Incoming Connections: When a server socket is in the listening state, it establishes a queue for incoming	4	L5	2	3	2.4.1

	connection requests. These connection requests are typically placed in a queue known as the "backlog." Backlog Size: The listen() function allows you to specify the size of the backlog, which is essentially the maximum number of pending connections that can be queued while the server is processing existing connections. The actual maximum number of pending connections a server can handle depends on the operating system and its configuration.					
	Part – B (3x10=30 Marks)					
8.a(i)	Explain in detail about Concurrent Programming Paradigm. Concurrent programming is a programming paradigm that focuses on writing software that can execute multiple tasks concurrently. Concurrency allows for more efficient and responsive software by taking advantage of modern multi-core processors and distributed systems. Concurrency is the ability of a system to handle multiple tasks at the same time. Concurrency can be achieved through techniques such as multi-threading, multi-processing, or asynchronous programming. Threads and Processes: Threads and processes are the basic units of concurrent programming. A thread is a lightweight unit of a process. Multiple threads can exist within a single process and share the same memory space, making thread communication and synchronization easier. A process is an independent program with its own memory space. Processes are typically used for more significant isolation and can communicate via inter-process communication (IPC) mechanisms. Synchronization: To ensure that concurrent threads or processes work correctly, synchronization mechanisms like locks, semaphores, and mutexes are used. These mechanisms prevent data races and ensure that shared resources are accessed safely. Shared Memory vs. Message Passing: In shared memory systems, threads or processes communicate by sharing memory. In message-passing systems, threads or processes communicate by passing messages. This approach is more isolated but can be less efficient due to message overhead. Benefits of Concurrent Programming: Improved Performance: Concurrent programs can take advantage of multiple CPU cores and execute tasks in parallel, leading to improved performance and	4	L3	2	3	2.1.2

	responsiveness. Responsiveness: Concurrency can make software more responsive, especially in user interfaces and networked applications, by allowing tasks to run concurrently without blocking the main thread or process. Resource Utilization: Concurrent programming can maximize resource utilization, making efficient use of CPU, memory, and other system resources. Scalability: Concurrent systems can scale to handle increased workloads, making them suitable for multi-user systems and server applications. Challenges of Concurrent Programming: Data Races: Data races occur when multiple threads or processes access shared data concurrently without proper synchronization, leading to unpredictable and erroneous behavior. Deadlocks: Deadlocks are situations where threads or processes are stuck waiting for resources that won't be released. They can bring the entire system to a standstill. Synchronization Overhead: Synchronization mechanisms add overhead to code execution, potentially reducing the performance gains of concurrency. Programming Languages for Concurrent Programming include Java, C++, Python and Go. These languages offer constructs for creating threads, managing synchronization, and implementing concurrent algorithms.					

a(ii)	Write a java program that implements a multi-thread application that has three threads. First thread generates random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd, the third thread will print the value of cube of the number.
	
	import java.util.Random;
	public class MultithreadExample {
	public static void main(String[] args) {
	RandomNumberGenerator generator =
	new RandomNumberGenerator();
	SquareCalculator squareThread =
	new SquareCalculator(generator);
	CubePrinter cubeThread =
	new CubePrinter(generator);
	
	generator.start();
	squareThread.start();
	cubeThread.start();
	}
	}
		6	L3	2	3	2.1.4

```
class RandomNumberGenerator extends Thread {
   private boolean running = true;

   @Override
   public void run() {
      Random random = new Random();
      while (running) {
         int randomNumber = random.nextInt(100);
         System.out.println("Generated:                "                +
randomNumber);

         if (randomNumber % 2 == 0) {
            SquareCalculator.newNumber(randomNumber);
         } else {
            CubePrinter.newNumber(randomNumber);
         }
         try {
            Thread.sleep(1000); // Sleep for 1 second
         } catch (InterruptedException e) {
            e.printStackTrace();
         }
      }
   }
   public void stopGenerating() {
      running = false;
   }
}

class SquareCalculator extends Thread {
   private static int number;

public SquareCalculator(RandomNumberGenerator
generator){
      generator.start();
   }


 public static void newNumber(int num) {
      number = num;
   }

   @Override
   public void run() {
      while (true) {
         if (number % 2 == 0) {
            int square = number * number;
            System.out.println("Square: " + square);
         }
      }
   }
}

class CubePrinter extends Thread {
   private static int number;

   public CubePrinter(RandomNumberGenerator generator)
{
      generator.start();
   }
```

	public static void newNumber(int num) {
	number = num;
	}
	
	@Override
	public void run() {
	while (true) {
	if (number % 2 != 0) {
	int cube = number * number * number;
	System.out.println("Cube: " + cube);
	}
	}
	}
	}					

	(OR)					
8.b(i)	Discuss the five steps for connecting JDBC. Connecting to a relational database using JDBC (Java Database Connectivity) typically involves five steps. JDBC is a Java-based API that allows Java applications to interact with relational databases. Here are the five steps for connecting to a database using JDBC: **Import Required Packages:** Import the necessary JDBC packages. The key package to import is java.sql, which contains the core JDBC classes and interfaces. import java.sql.*; **Load and Register the JDBC Driver:** JDBC drivers are platform-specific implementations that allow Java to communicate with the database. To use a specific database, you need to load and register the appropriate JDBC driver. This step is usually performed using the Class.forName() method or driver-specific methods. Class.forName("com.mysql.cj.jdbc.Driver"); **Establish a Database Connection:** Use the DriverManager class to establish a connection to your database. You need to provide the database URL, username, and password as parameters to the getConnection() method. String url = "jdbc:mysql://localhost:3306/mydatabase"; String username = "your_username"; String password = "your_password"; Connection connection =DriverManager.getConnection(url, username, password); **Execute SQL Queries and Process Results:** To interact with the database, you need to create a Statement or PreparedStatement object. A Statement is used for executing simple SQL queries, while a	4	L3	2	2	1.4.1

	PreparedStatement is used for parameterized queries and offers better security against SQL injection.
	
	PreparedStatement preparedstatement =
	
	connection.prepareStatement("SELECT * FROM mytable WHERE id = ?");
	
	Use the execute(), executeQuery(), or executeUpdate() methods of the Statement or PreparedStatement to execute SQL queries against the database.
	
	ResultSet resultSet = statement.executeQuery("SELECT * FROM mytable");
	
	// Process the result set
	
	while (resultSet.next()) {
	
	//Retrieve data using resultSet.getXXX() methods
	
	int id = resultSet.getInt("id");
	
	String name = resultSet.getString("name"); }
	
	Close the Connection and Resources: After you're done with the database operations, it's essential to close the database connection and any associated resources, such as Statement, PreparedStatement, and ResultSet objects, to release resources and ensure proper clean-up.
	
	resultSet.close();
	
	statement.close();
	
	connection.close();
	
	These five steps outline the fundamental process for connecting to a database using JDBC in Java.					

b(ii)	Create a java program for the following
	
	Use ThreadA to find number of digits present in the string k and store into variable dc, finally print the value of dc(output format: ThreadA:digitscount). Use ThreadB to find number of alphabetic present in the string k and store into variable cc, finally print the value of cc (output format:ThreadB:digitscount).
	
	public class CharacterCount {
	private static String k = "Hello123World456"; // Replace with your input string
	
	public static void main(String[] args) {
	ThreadA threadA = new ThreadA();
	ThreadB threadB = new ThreadB();
	
	threadA.start();
	threadB.start();
	
	try {
		6	L3	1	1	1.2.1

	threadA.join();
	threadB.join();
	} catch (InterruptedException e) {
	e.printStackTrace();
	}
	
	int dc = threadA.getDigitCount();
	int cc = threadB.getCharacterCount();
	
	System.out.println("ThreadA:" + dc);
	System.out.println("ThreadB:" + cc);
	}
	}
	
	class ThreadA extends Thread {
	private int digitCount = 0;
	
	@Override
	public void run() {
	for (char c : CharacterCount.k.toCharArray()) {
	if (Character.isDigit(c)) {
	digitCount++;
	}
	}
	}
	
	public int getDigitCount() {
	return digitCount;
	}
	}
	
	class ThreadB extends Thread {
	private int characterCount = 0;
	
	@Override
	public void run() {
	for (char c : CharacterCount.k.toCharArray()) {
	if (Character.isLetter(c)) {
	characterCount++;
	}
	}
	}
	
	public int getCharacterCount() {
	return characterCount;
	}
	}					

9.a(i)	Write a program to implement First Order Logic. from pyke import knowledge_engine # Define a simple knowledge base with facts and rules. facts = [("man", "John"), ("woman", "Mary"),	4	L4	1	2	2.1.1

	("parent", "John", "Tom"), ("parent", "Mary", "Tom"),] engine = knowledge_engine.engine(__file__) with engine.prove_goal("man($x)") as gen: for vars, plan in gen: print(f"{vars['x']} is a man.") with engine.prove_goal("parent($x, Tom)") as gen: for vars, plan in gen: print(f"{vars['x']} is a parent of Tom.")					

a(ii)	Write a python program to Check if a Number is Even or Odd and also check whether it is Prime or not.
	
	def is_even_or_odd(number):
	if number % 2 == 0:
	return "Even"
	else:
	return "Odd"
	
	def is_prime(number):
	if number <= 1:
	return False
	if number <= 3:
	return True
	if number % 2 == 0 or number % 3 == 0:
	return False
	i = 5
	while i * i <= number:
	if number % i == 0 or number % (i + 2) == 0:
	return False
	i += 6
	return True
	
	# Input a number from the user
	num = int(input("Enter a number: "))
	
	# Check if the number is even or odd
	even_or_odd = is_even_or_odd(num)
	print(f"{num} is {even_or_odd}.")
	
	# Check if the number is prime or not
	prime = is_prime(num)
	if prime:
	print(f"{num} is a prime number.")
	else:
	print(f"{num} is not a prime number.")
		6	L4	2	1	2.1.1

	(OR)					
9.b(i)	Illustrate the prolog features with examples. Prolog is a powerful logic programming language used for	4	L4	3	1	1.2.1

	knowledge representation, rule-based reasoning, and symbolic reasoning. It's designed to work with facts and rules that express relationships and logical implications. Here are some key Prolog features illustrated with examples: Facts and Rules: Prolog allows you to define facts and rules using predicates. Facts are statements that are always true, while rules define relationships and conditions. Here's an example: % Facts likes(john, pizza). likes(mary, pizza). likes(mary, ice_cream). % Rule eats(john, X) :- likes(john, X). In this example, we have facts about who likes what and a rule that defines what John eats based on his likes. Querying and Inference: Prolog allows you to query the knowledge base to make inferences. For example: ?- likes(john, pizza). true. ?- eats(john, ice_cream). false. Prolog can determine that John likes pizza (based on the facts) and that John doesn't eat ice cream (because it's not defined in the facts or rules). Variables: Prolog uses variables to represent unknown values. You can use variables in queries and rules. For example: ?- likes(X, pizza). X = john ; X = mary. Prolog can find all individuals who like pizza by using the variable X. Negation: Prolog allows you to express negation through the not or \+ operator. For example:					

	% Rule for dislikes dislikes(Person, Food) :- \+ likes(Person, Food). ?- dislikes(john, ice_cream). true. The dislikes rule is defined as the negation of the likes rule. Lists: Prolog supports lists and provides predicates for list manipulation. For example: % Recursive rule to calculate the sum of a list of numbers sum_list([], 0). sum_list([Head | Tail], Sum) :- sum_list(Tail, RestSum), Sum is Head + RestSum. ?- sum_list([1, 2, 3, 4], Sum). Sum = 10. Recursion: Prolog is well-suited for recursive definitions. Here's an example of a recursive rule to find the factorial of a number: factorial(0, 1). factorial(N, Result) :- N > 0, N1 is N - 1, factorial(N1, PartialResult), Result is N * PartialResult. You can calculate the factorial of a number using this rule. Prolog is widely used in artificial intelligence, expert systems, and natural language processing due to its declarative and rule-based nature.					

b(ii)	Write a python program to calculate the area and perimeter of different geometric shapes (circle, rectangle, triangle, etc.).
	
	import math
	
	# Function to calculate the area and perimeter of a circle
	def circle_area_and_perimeter(radius):
	  area = math.pi * radius**2
	  perimeter = 2 * math.pi * radius
	  return area, perimeter
	
	# Function to calculate the area and perimeter of a rectangle
	
	def rectangle_area_and_perimeter(length, width):
	  area = length * width
	  perimeter = 2 * (length + width)
	  return area, perimeter
		6	L4	3	2	2.1.3

	# Function to calculate the area and perimeter of a triangle
	def triangle_area_and_perimeter(base, height, side1, side2, side3):
	  area = 0.5 * base * height
	  perimeter = side1 + side2 + side3
	  return area, perimeter
	
	# Example usage
	shape = input("Enter the shape (circle, rectangle, triangle): ")
	
	if shape == "circle":
	  radius = float(input("Enter the radius: "))
	  area, perimeter = circle_area_and_perimeter(radius)
	elif shape == "rectangle":
	  length = float(input("Enter the length: "))
	  width = float(input("Enter the width: "))
	  area, perimeter = rectangle_area_and_perimeter(length, width)
	elif shape == "triangle":
	  base = float(input("Enter the base length: "))
	  height = float(input("Enter the height: "))
	  side1 = float(input("Enter the length of side 1: "))
	  side2 = float(input("Enter the length of side 2: "))
	  side3 = float(input("Enter the length of side 3: "))
	  area, perimeter = triangle_area_and_perimeter (base, height, side1, side2, side3)
	else:
	  print("Unsupported shape")
	
	if shape in ["circle", "rectangle", "triangle"]:
	  print(f"Area: {area}")
	  print(f"Perimeter: {perimeter}")					

10.a(i)	How can you use third-party libraries with Tkinter to enhance its functionality? How do you install Tkinter? Tkinter is a standard library for building graphical user interfaces (GUIs) in Python. While Tkinter provides a basic set of widgets and functionality for creating GUI applications, you can enhance its functionality by integrating third-party libraries. These libraries can add features, custom widgets, and additional capabilities to your Tkinter-based applications. Here are the steps to use third-party libraries with Tkinter: Install Tkinter: Tkinter is included in Python's standard library, so you typically do not need to install it separately. To ensure you have Tkinter available, make sure you have Python installed on your system.	4	L5	1	3	1.4.1

	Install Third-Party Libraries: To enhance Tkinter's functionality, you can install third-party libraries using package managers like pip. For example: pip install library_name Import the Library: In your Python code, import the third-party library that you've installed. For example: import library_name Use the Library's Features: Utilize the features provided by the third-party library to enhance your Tkinter application. This may involve creating custom widgets, adding new functionality, or modifying the appearance of your GUI. Integrate with Tkinter: Depending on the library, you may need to integrate its features with your Tkinter application. This often involves creating Tkinter widgets and binding events to handle interactions with the library's components. Customization and Styling: Some third-party libraries allow you to customize the appearance and style of your GUI components. You can use these features to create a more appealing and user-friendly interface. Here are a few third-party libraries that are commonly used in conjunction with Tkinter to enhance its functionality: ttkthemes: This library provides additional themed Tkinter widgets and allows you to customize the appearance of your GUI components with various themes. Pillow (PIL): The Python Imaging Library (PIL) or Pillow allows you to work with images and perform image processing within your Tkinter application. Matplotlib: Matplotlib is a popular library for creating data visualizations, including charts and plots. You can embed Matplotlib figures within your Tkinter application. ttkwidgets: This library offers additional themed Tkinter widgets, enhancing the default set of widgets provided by Tkinter. Each library may have its own documentation and usage instructions, so refer to the library's documentation for specific details on how to use its features in your Tkinter					

	application.					
a(ii)	Construct a NFA and give the transmission table for the following regular expression for the given language = (a + ba)*bb (a + ab)*. To construct a Non-Deterministic Finite Automaton (NFA) for the regular expression `(a + ba)*bb (a + ab)*`, we can break down the expression into two parts: `(a + ba)*` and `(a + ab)*`. Each part can be implemented as a separate NFA, and then we can combine them. Let's create NFAs for each part and then combine them: 1. NFA for `(a + ba)*`: - State q0: Initial state - State q1: Accepting state for `(a + ba)*` - Transitions: - q0 on 'a' goes to q0 (loop for 'a') - q0 on 'b' goes to q1 (transition for 'ba') - q1 on 'a' goes to q0 (loop for 'a') - q1 on 'b' goes to q1 (stay for 'ba') 2. NFA for `(a + ab)*`: - State q2: Initial state - State q3: Accepting state for `(a + ab)*` - Transitions: - q2 on 'a' goes to q3 (transition for 'a') - q2 on 'b' goes to q2 (loop for 'b') - q3 on 'a' goes to q3 (stay for 'ab') - q3 on 'b' goes to q3 (stay for 'ab') Now, we can combine these two NFAs into a single NFA: 3. Combined NFA for `(a + ba)*bb (a + ab)*`: - State q0: Initial state - State q1: Accepting state for `(a + ba)*` - State q2: Initial state for `(a + ab)*` - State q3: Accepting state for `(a + ab)*` - Transitions: - q0 on 'a' goes to q0 (loop for `(a + ba)*`) - q0 on 'b' goes to q1 (transition for `(a + ba)*`) - q1 on 'b' goes to q1 (stay for `(a + ba)*`) - q1 on 'a' goes to q0 (loop for `(a + ba)*`) - q2 on 'a' goes to q3 (transition for `(a + ab)*`) - q2 on 'b' goes to q2 (loop for `(a + ab)*`) - q3 on 'a' goes to q3 (stay for `(a + ab)*`) - q3 on 'b' goes to q3 (stay for `(a + ab)*`) Transmission Table for the Combined NFA: | State | 'a' | 'b' | |-------|------|------| | q0 | q0 | q1 |	6	L5	1	2	2.1.2

	| q1 | q0 | q1 |
	| q2 | q3 | q2 |
	| q3 | q3 | q3 |
	
	This NFA recognizes strings that match the regular expression `(a + ba)*bb (a + ab)*`. It can be used to determine if a given string belongs to the specified language.
	
	In python, we implement this as follows -
	
	`# Define the NFA transitions using a dictionary`
	`nfa = {`
	`  0: {'a': {0}, 'b': {1}},`
	`  1: {'b': {1}},`
	`  2: {'a': {3}, 'b': {2}},`
	`  3: {'a': {3}},`
	`}`
	`# Define the set of accepting states`
	`accepting_states = {1, 3}`
	
	`# Function to simulate the NFA`
	`def simulate_nfa(input_string):`
	`  current_states = {0}`
	`  for char in input_string:`
	`  next_states = set()`
	`  for state in current_states:`
	`  if char in nfa[state]:`
	`  next_states.update(nfa[state][char])`
	`  current_states = next_states`
	
	`  return any(state in current_states for state in accepting_states)`
	
	`# Test the NFA with some strings`
	`test_strings = ["bb", "abb", "abab", "baa", "aabbaab"]`
	`for string in test_strings:`
	`  if simulate_nfa(string):`
	`  print(f"'{string}' is in the language")`
	`  else:`
	`  print(f"'{string}' is not in the language")`					

	(OR)					
10.b(i)	What functions are available for solving equations in SymPy? SymPy is a Python library for symbolic mathematics, and it provides a wide range of functions and tools for solving equations symbolically. Here are some of the key functions available for solving equations in SymPy: A. solve Function: The `solve` function is the primary tool for solving equations in SymPy. It can solve equations and systems of equations symbolically, including algebraic, transcendental, and differential equations. Example: `  from sympy import symbols, Eq, solve`	4	L5	3	2	1.2.1

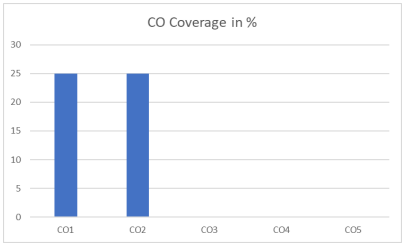
	x, y = symbols('x y') equation = Eq(x**2 + y, 5) solutions = solve(equation, x) B. solveset Function: The `solveset` function finds solutions to equations, inequalities, and systems of equations. It returns a `FiniteSet`, `Interval`, `Union`, or other set representations of the solutions. Example: from sympy import symbols, Eq, solveset x = symbols('x') equation = Eq(x**2 - 4, 0) solutions = solveset(equation, x, domain=S.Reals) C. linsolve Function: The `linsolve` function is specifically designed for solving systems of linear equations. It returns solutions in the form of a tuple of expressions or as a set. Example: from sympy import symbols, linsolve x, y = symbols('x y') system = (2*x + 3*y - 5, 4*x - 6*y - 2) solutions = linsolve(system, x, y) D. dsolve Function: The `dsolve` function is used for solving ordinary differential equations (ODEs). It can handle a wide range of differential equations and return the symbolic solutions. Example: python from sympy import symbols, Eq, Function, dsolve x = symbols('x') f = Function('f')(x) ode = Eq(f.diff(x) - 2*f, 0) solution = dsolve(ode, f) These are some of the fundamental functions in SymPy for solving equations, inequalities, and differential equations. SymPy's symbolic math capabilities make it a powerful tool for algebraic manipulation and solving mathematical problems.					

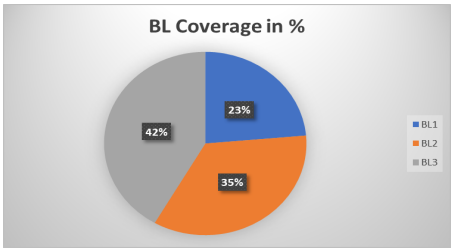
b(ii)	Develop a Python function that takes an NFA and returns the set of states that can be reached from a given state on a specific input symbol.
	
	We can implement a depth-first search (DFS) algorithm. The DFS algorithm will explore the transitions of the NFA to find all reachable states on the given input symbol.
	
	Here's a Python function that accomplishes this:
	
	def find_reachable_states(nfa, start_state, input_symbol):
	def dfs(current_state):
	# Initialize the set of reachable states with the current state.
		e	L5	3	2	2.1.4

	reachable_states = {current_state}
	
	# Get the transitions for the current state and input symbol.
	transitions = nfa.get(current_state, {}).get(input_symbol, set())
	
	# Explore each transition recursively.
	for next_state in transitions:
	# Perform a depth-first search from the next state.
	reachable_states.update(dfs(next_state))
	
	return reachable_states
	
	# Start the DFS from the given start state.
	reachable_states = dfs(start_state)
	return reachable_states
	
	nfa = {
	'q0': {'a': {'q0', 'q1'}, 'b': {'q0'}},
	'q1': {'a': {'q1'}, 'b': {'q2'}},
	'q2': {'a': {'q2'}, 'b': {'q2'}}
	}
	
	start_state = 'q0'
	input_symbol = 'a'
	
	reachable_states = find_reachable_states(nfa, start_state, input_symbol)
	print("Reachable states:", reachable_states)
	
	In this example, the find_reachable_states function takes the NFA represented as a dictionary, the start state, and the input symbol. It uses a recursive DFS algorithm to find all reachable states starting from the given state on the specified input symbol. The reachable states are stored in a set, and the function returns this set.					

***Program Indicators are available separately for Computer Science and Engineering in AICTE examination reforms policy.**

Course Outcome (CO) and Bloom's level (BL) Coverage in Questions





Approved by the Audit Professor/Course Coordinator